

Введение и цели работы

В этой лабораторной работе вы освоите технику **Transfer Learning** – перенос знаний из одной задачи машинного обучения в другую. Это позволяет быстро и эффективно решать задачи классификации, детекции объектов и семантической сегментации, используя уже обученные на больших данных модели.

Задачи:

- Научиться загружать и использовать предобученные модели в PyTorch.
- Адаптировать архитектуру под новую задачу (замена финальных слоёв).
- Проводить обучение и оценку моделей с помощью трансферного обучения.
- Анализировать метрики качества и интерпретировать результаты.

2. Подготовка окружения

2.1. Установка PyTorch и необходимых библиотек

```
bash
pip install torch torchvision torchaudio tqdm matplotlib
```

Для работы с изображениями и визуализацией:

```
bash
pip install pillow matplotlib
```

Для задач детекции и сегментации:

```
bash
pip install torchvision
```

2.2. Проверка установки

```
python
import torch
print(torch.__version__)
```

3. Теоретическая часть: Transfer Learning

Transfer Learning – это подход, при котором веса нейросети, обученной на одной задаче (например, ImageNet), используются для решения другой, часто более узкой задачи.

Преимущества:

- Сокращение времени обучения.
- Меньшая потребность в данных.
- Повышение точности на малых выборках.

Типовые шаги:

1. Загрузка предобученной модели.
2. Замена финального слоя под количество классов вашей задачи.
3. Заморозка части слоёв (опционально).
4. Обучение на новом датасете.

4. Варианты выполнения работы

4.1. Вариант 1: Классификация изображений

4.1.1. Выбор датасета и модели

Выберите один из предложенных датасетов (например, MNIST, Fashion-MNIST, STL-10, Oxford Flowers 102 и др.) и архитектуру (ResNet, VGG, MobileNet).

4.1.2. Пример кода для классификации

```

import torch
import torch.nn as nn
import torchvision.models as models
import torchvision.transforms as T
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

# Загрузка предобученной модели (например, ResNet18)
model = models.resnet18(pretrained=True)

# Замена финального слоя под количество классов вашего датасета
num_classes = 10 # например, для Fashion-MNIST
model.fc = nn.Linear(model.fc.in_features, num_classes)

# Опционально: заморозка слоёв
for param in model.parameters():
    param.requires_grad = False
for param in model.fc.parameters():
    param.requires_grad = True

# Определение преобразований для данных
transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Загрузка датасета
train_dataset = ImageFolder('data/train', transform=transform)
val_dataset = ImageFolder('data/val', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32)

# Определение функции потерь и оптимизатора
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)

# Пример цикла обучения (упрощённо)
for epoch in range(5):
    model.train()
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
    print(f'Epoch {epoch+1}, Loss: {loss.item()}')

```

4.1.3. Оценка метрик

Используйте стандартные метрики:

- **Accuracy** (доля правильных ответов).
- **Precision, Recall, F1-score** (для несбалансированных классов).
- **Confusion Matrix** (визуализация ошибок).

4.2. Вариант 2: Детекция объектов (Object Detection)

4.2.1. Выбор датасета и архитектуры

Датасеты: Pascal VOC, COCO, Open Images. Архитектуры: Faster R-CNN, SSD, YOLO, DETR.

4.2.2. Пример кода для детекции (Faster R-CNN)

```

import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

```

```
# Загрузка предобученной модели Faster R-CNN с ResNet-50-FPN
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)

# Замена классификационного слоя под количество классов (например, +1 для фона)
num_classes = 21 # для Pascal VOC: 20 классов + фон
in_features = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
```

Далее – загрузка датасета (например, через `torchvision.datasets.VOCDetection`), настройка `DataLoader`, обучение и оценка по метрикам mAP (mean Average Precision).

4.3. Вариант 3: Семантическая сегментация

4.3.1. Выбор датасета и архитектуры

Датасеты: Cityscapes, ADE20k, CamVid. Архитектуры: U-Net, DeepLabV3, PSPNet.

4.3.2. Пример кода для сегментации (DeepLabV3)

```
import torchvision
from torchvision.models.segmentation import deeplabv3_resnet50

model = deeplabv3_resnet50(pretrained=True)
num_classes = 21 # например, для Pascal VOC Segmentation
model.classifier[4] = nn.Conv2d(256, num_classes, kernel_size=(1, 1), stride=(1, 1))
```

Далее – загрузка датасета (например, `torchvision.datasets.VOCSegmentation`), обучение и оценка по IoU (Intersection over Union).

5. Оформление отчёта

В отчёте должны быть:

1. Описание выбранной задачи и датасета.
2. Описание архитектуры и изменений в ней.
3. Графики обучения (loss/accuracy).
4. Таблица с метриками (accuracy, F1, mAP, IoU).
5. Примеры предсказаний модели (визуализация).
6. Выводы по результатам работы.

6. Полезные ресурсы

- PyTorch Tutorials
- torchvision models
- Официальные датасеты

1. Введение и теоретические основы

Transfer Learning (перенос обучения) – это подход, при котором используются веса нейросети, обученной на одной задаче (например, ImageNet), для решения другой, часто более узкой задачи. Такой метод позволяет:

- значительно ускорить обучение;
- добиться высокой точности даже при небольшом объёме данных;
- снизить требования к вычислительным ресурсам.

В задачах классификации изображений обычно используют предобученные модели (ResNet, VGG, MobileNet и др.), заменяя их финальный слой на новый, соответствующий количеству классов в вашем датасете.

2. Подготовка окружения

2.1. Установка необходимых библиотек

```
bash
pip install torch torchvision tqdm matplotlib pillow
```

2.2. Проверка установки

```
import torch
print(torch.__version__)
```

3. Загрузка и подготовка данных

3.1. Организация датасета

Структура папок для датасета:

```
dataset/
├── train/
│   ├── class_1/
│   ├── class_2/
│   └── ...
└── val/
    ├── class_1/
    ├── class_2/
    └── ...
```

3.2. Загрузка данных с помощью ImageFolder

```
import torchvision.transforms as T
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader

transform = T.Compose([
    T.Resize((224, 224)),          # Приведение к размеру модели
    T.RandomHorizontalFlip(),      # Аугментация
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

train_dataset = ImageFolder('dataset/train', transform=transform)
val_dataset   = ImageFolder('dataset/val',   transform=transform)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset,   batch_size=32)
```

4. Загрузка и модификация предобученной модели

4.1. Пример с ResNet18

```
import torchvision.models as models
import torch.nn as nn

# Загрузка предобученной модели
model = models.resnet18(pretrained=True)

# Замена финального слоя под количество классов вашего датасета
num_classes = len(train_dataset.classes)
model.fc = nn.Linear(model.fc.in_features, num_classes)
```

4.2. Заморозка слоёв (опционально)

```
for param in model.parameters():
    param.requires_grad = False # Заморозка всех слоёв

for param in model.fc.parameters():
    param.requires_grad = True # Только финальный слой обучается
```

Альтернатива: разморозить часть слоёв для fine-tuning.

5. Определение функции потерь и оптимизатора

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Если размораживаете больше слоёв – передавайте в оптимизатор model.parameters().

6. Цикл обучения и валидации

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = model.to(device)
```

```
num_epochs = 10
```

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    print(f'Epoch {epoch+1}, Loss: {running_loss/len(train_loader)}')

    # Валидация
    model.eval()
    correct = total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()
    print(f'Validation Accuracy: {100 * correct / total:.2f}%')
```

7. Оценка качества модели

7.1. Основные метрики

- **Accuracy** – доля правильных предсказаний.
- **Precision, Recall, F1-score** – для несбалансированных классов.
- **Confusion Matrix** – визуализация ошибок.

7.2. Пример вычисления метрик

```
from sklearn.metrics import classification_report, confusion_matrix

model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for images, labels in val_loader:
        images = images.to(device)
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.numpy())

print(classification_report(all_labels, all_preds))
print(confusion_matrix(all_labels, all_preds))
```

8. Визуализация результатов

8.1. Пример визуализации предсказаний

```
import matplotlib.pyplot as plt
import numpy as np

def imshow(img):
    img = img / 2 + 0.5 # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()

dataiter = iter(val_loader)
images, labels = next(dataiter)
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % train_dataset.classes[labels[j]] for j
in range(4)))

outputs = model(images.to(device))
_, predicted = torch.max(outputs, 1)
print('Predicted: ', ' '.join('%5s' % train_dataset.classes[predicted[j]] for
j in range(4)))
```

9. Сохранение и загрузка модели

```
torch.save(model.state_dict(), 'best_model.pth')
# Загрузка:
model.load_state_dict(torch.load('best_model.pth'))
```

10. Рекомендации по улучшению результата

- Используйте data augmentation (RandomCrop, ColorJitter и др.).
- Экспериментируйте с разными архитектурами (ResNet50, EfficientNet).
- Fine-tune: размораживайте больше слоёв и обучайте с меньшим learning rate.
- Применяйте learning rate schedulers.
- Анализируйте confusion matrix для выявления сложных классов.

Ключевые отличия Transfer Learning в задачах детекции объектов (Object Detection) по сравнению с классификацией изображений

В задаче детекции объектов подход к переносу обучения (Transfer Learning) имеет ряд принципиальных отличий от классической классификации. Вот подробный разбор изменений на каждом этапе.

1. Структура данных

- **Классификация:** каждый образец – это одно изображение с одной меткой (класс).
- **Детекция:** каждое изображение сопровождается аннотациями в формате *bounding box* (координаты прямоугольника) и меткой класса для каждого объекта на изображении.

2. Архитектуры моделей

- **Классификация:** используются стандартные *CNN* (ResNet, VGG, MobileNet и др.).
- **Детекция:** применяются специализированные архитектуры, такие как:
 - *Faster R-CNN*,
 - *SSD*,
 - *YOLO*,
 - *DETR*,
 - *RetinaNet*.

Эти модели состоят из двух основных частей:

- **Backbone** – предобученная *CNN* для извлечения признаков (например, ResNet).
- **Head** – блоки, отвечающие за предсказание рамок (*region proposal*), классификацию объектов и уточнение координат.

3. Процесс переноса обучения

- **Классификация:** заменяется только финальный полносвязный слой (*fc*) на новый, соответствующий числу классов.
- **Детекция:**
 - Загружается предобученная модель (обычно на *COCO* или *ImageNet*).
 - Заменяется классификационный head (например, `box_predictor` в *Faster R-CNN*) на новый, соответствующий числу классов вашей задачи.
 - Иногда замораживается backbone и обучается только head; при необходимости размораживаются и нижние слои для тонкой настройки (*fine-tuning*).

4. Подготовка данных

- **Классификация:** достаточно организовать папки по классам и использовать `ImageFolder`.

- **Детекция:**
 - Требуются аннотации в специальных форматах: *COCO JSON*, *Pascal VOC XML*, *CSV*.
 - Используются специальные датасеты и загрузчики, например: `dataset = torchvision.datasets.VOCDetection(...)`
 - Применяются преобразования, поддерживающие как изображение, так и аннотации (*transforms*).

5. Цикл обучения

- **Классификация:** стандартный цикл с прямым проходом, вычислением кросс-энтропии и обновлением весов.
- **Детекция:**
 - Модель возвращает не только предсказанные классы, но и координаты рамок (*bounding boxes*).
 - Функция потерь включает:
 - классификационную потерю (например, *cross-entropy*);
 - регрессионную потерю для координат рамок (например, *smooth L1 loss*);
 - иногда дополнительные компоненты (например, для балансировки классов).
 - В цикле обучения обрабатываются как изображения, так и их аннотации.

6. Оценка качества

- **Классификация:** используются метрики accuracy, precision, recall, F1.
- **Детекция:**
 - Основной метрикой является *mAP* (mean Average Precision).
 - Дополнительно анализируются:
 - *IoU* (Intersection over Union) – степень перекрытия предсказанной и истинной рамки;
 - precision/recall по классам;
 - визуализация предсказаний с рамками.

7. Визуализация результатов

- **Классификация:** выводится предсказанный класс для изображения.
- **Детекция:** на изображении рисуются рамки вокруг обнаруженных объектов с подписями классов и вероятностями.

Краткое резюме изменений

Этап	Классификация	Детекция объектов
Данные	Одно изображение + класс	Изображение + рамки + классы
Архитектура	CNN	Backbone + Head (RPN, регрессия)

Перенос обучения	Замена fc-слоя	Замена head, иногда fine-tuning backbone
Потери	CrossEntropy	CrossEntropy + регрессия рамок
Метрики	Accuracy, F1	mAP, IoU
Визуализация	Класс изображения	Рамки + классы на изображении

Ключевые отличия Transfer Learning для семантической сегментации

В задаче семантической сегментации подход к переносу обучения (Transfer Learning) существенно отличается как от классификации, так и от детекции объектов.

1. Структура данных

- **Классификация:** одно изображение – одна метка (класс).
- **Детекция:** изображение + аннотации в виде прямоугольников (bounding boxes) и классов.
- **Сегментация:** изображение + **маска сегментации** (разметка каждого пикселя классом объекта). Маска обычно имеет тот же размер, что и изображение, и содержит целочисленные значения – индексы классов.

2. Архитектуры моделей

- **Классификация:** стандартные *CNN* (ResNet, VGG, MobileNet).
- **Детекция:** архитектуры с *region proposal* (Faster R-CNN, YOLO, SSD).
- **Сегментация:** специализированные архитектуры, которые строят попиксельные предсказания:
 - *U-Net*,
 - *FCN* (Fully Convolutional Network),
 - *DeepLabV3*,
 - *PSPNet*,
 - *HRNet*,
 - *SegNet*.

В таких моделях часто используется предобученный backbone (например, ResNet), а затем добавляется декодер (или head), который восстанавливает пространственное разрешение до исходного размера изображения.

3. Процесс переноса обучения

- **Классификация:** заменяется только финальный полносвязный слой (*fc*).
- **Детекция:** заменяется классификационный head (например, `box_predictor`).
- **Сегментация:**
 - Загружается предобученная модель (обычно на ImageNet).

- Заменяется финальный классификационный слой на новый, соответствующий числу классов вашей задачи.
- В некоторых архитектурах (например, DeeperLabV3) заменяется только head, а backbone остаётся без изменений.
- Возможна полная разморозка backbone для тонкой настройки (*fine-tuning*).

4. Подготовка данных

- **Классификация:** папки по классам, `ImageFolder`.
- **Детекция:** аннотации в формате *COCO JSON*, *Pascal VOC XML* и др.
- **Сегментация:**
 - Используются специальные датасеты с масками (Cityscapes, ADE20k, Pascal VOC Segmentation).
 - Применяются специальные датасеты и загрузчики, например: `dataset = torchvision.datasets.VOCSegmentation(...)`
 - Преобразования (*transforms*) применяются одновременно к изображению и маске (например, случайные повороты, масштабирование).

5. Цикл обучения

- **Классификация:** стандартный цикл с прямым проходом и кросс-энтропией.
- **Детекция:** комбинированная потеря (классификация + регрессия рамок).
- **Сегментация:**
 - Модель возвращает карту классов того же размера, что и входное изображение.
 - Основная функция потерь – **кросс-энтропия для попиксельной классификации**.
 - В некоторых случаях используются дополнительные потери (например, Dice Loss для несбалансированных классов).

6. Оценка качества

- **Классификация:** accuracy, precision, recall, F1.
- **Детекция:** mAP, IoU по рамкам.
- **Сегментация:**
 - Основной метрикой является **mIoU** (mean Intersection over Union) – среднее значение IoU по всем классам.
 - Дополнительно анализируются:
 - попиксельная точность;
 - визуализация предсказанных масок.

7. Визуализация результатов

- **Классификация:** выводится класс изображения.
- **Детекция:** рамки с классами на изображении.
- **Сегментация:** визуализируется наложение предсказанной маски на исходное изображение (часто с цветовой кодировкой классов).

Краткое резюме изменений

Этап	Классификация	Детекция	Сегментация
Данные	Изображение + класс	Изображение + рамки + классы	Изображение + попиксельная маска
Архитектура	CNN	Backbone + RPN/Head	Backbone + Decoder/Head
Перенос обучения	Замена fc-слоя	Замена head	Замена финального слоя/head
Потери	CrossEntropy	CrossEntropy + регрессия рамок	Попиксельная CrossEntropy (+ Dice)
Метрики	Accuracy, F1	mAP, IoU по рамкам	mIoU
Визуализация	Класс изображения	Рамки + классы	Маска на изображении